REPUBLIQUE DU CAMEROUN
****
PAIX-TRAVAL-PATRIE
****
UNIVERSITE DE DSCHAN
****
FACULTES DES SCIENCES
****
DEPARTEMENT DE SCIENCES

REPUBLIC OF CAMEROON
****
PEACE-WORK-FATHERLAND
****
UNIVERSITY OF DSCHANG
****
FACULTY OF SCIENCES
****
DEPARTEMENT OF SCIENCES

Architecture des Systèmes Intelligents

Travaux Pratiques : Combinaison de classeurs, explicabilité des modèles

Présenté par :

DNJOMOU YONMBA WILFRIED LOIC	**CM-UDS-24SCI0999**
KENFACK LEONEL	**CM-UDS-17SCI0998**
SAGUEU WAKAM DILANE	**CM-UDS-24SCI1040**
NZOGNING MBONDA SOCRATE	**CM-UDS-19SCI2910**

Superviseur : Pr. KENGNE TCHENDJI Vianney

Année académique 2025-2026

Table des matières

1 État de l'art . . . 6
  1.1 Positionnement de notre étude . . . 7
2 Prétraitement et visualisation des données . . . 8
  2.1 Chargement et nettoyage . . . 8
  2.2 Gestion des valeurs manquantes et encodage . . . 8
  2.3 Visualisation des données . . . 9
  2.4 Découpage et équilibrage . . . 11
3 Interprétabilité intrinsèque des modèles et explicabilité post-hoc . . . 12
  3.1 Interprétabilité intrinsèque . . . 12
  3.2 Explicabilité post-hoc des modèles opaques . . . 14
  3.3 Synthèse . . . 17
4 Explicabilité des modèles de combinaison (bagging et boosting) . . . 18
  4.1 Modèles construits . . . 18
  4.2 TreeSHAP appliqué à la forêt aléatoire . . . 18
  4.3 Distillation de connaissances . . . 20
  4.4 Interactions SHAP appliquées à XGBoost . . . 21
  4.5 Tableau de gouvernance . . . 24
  4.6 Synthèse . . . 25
5 Approches avancees d'explicabilite : Stacking et XStacking . . . 26
  5.1 Stacking conventionnel avec validation croisee . . . 26
    Principe du Stacking . . . 26
    Configuration des modeles de base . . . 26
    Construction du Stacking . . . 27
    Resultats . . . 28
    Explicabilite du meta-apprenant . . . 28
    Tracabilite d'une prediction individuelle . . . 28
  5.2 XStacking : Stacking enrichi avec SHAP . . . 29
    Motivation : Le probleme de la perte de tracabilite . . . 29
    Paradigme XStacking . . . 29
    Calcul des valeurs SHAP . . . 30
    Analyse des importances SHAP . . . 31
    Construction du dataset XStacking . . . 33
    Entrainement et evaluation . . . 33
  5.3 Comparaison Stacking vs XStacking . . . 34

Analyse multi-dimensionnelle 34
Trade-off performance vs explicabilite 35
5.4 Insights cliniques et validation 36
Variables critiques identifiees 36
Validation medicale 36
5.5 Avantages et limites 37
Avantages du paradigme XStacking 37
Limites 37
5.6 Synthese 37

Table des figures

2.1 Distributions des variables numériques du jeu CKD (après imputation). 9
2.2 Effectifs des variables catégorielles (avant encodage). 10
2.3 Matrice de corrélation (triangle inférieur) entre toutes les variables du jeu CKD. . 10
3.1 Coefficients de la régression logistique (effet sur la prédiction CKD). 13
3.2 Arbre de décision (max_depth=4) pour la prédiction CKD. 14
3.3 Poids LIME pour une observation de test : contribution de chaque variable à la probabilité CKD. 15
3.4 Valeurs SHAP (explication locale) pour une observation de test. 16
3.5 Résumé SHAP (vue globale) : impact des variables sur la prédiction CKD. 17
4.1 Valeurs SHAP agrégées sur le jeu de test — forêt aléatoire. 19
4.2 Arbre de décision distillé depuis la forêt aléatoire (profondeur max. 4). 20
4.3 Graphique de dépendance SHAP (XGBoost). Axe des abscisses : taux d'hémoglobine (g/dL). Couleur : rouge = diabétique (dm = 1), bleu = non diabétique (dm = 0). 22
4.4 Interaction clinique : Hémoglobine × Diabète sucré — effet non linéaire et modulé de l'hémoglobine sur le risque de CKD (XGBoost). Axe des abscisses : taux d'hémoglobine (g/dL). Couleur : `diabetes_mellitus` (bleu = faible, rouge = élevé). 24
5.1 Importance des modeles de base dans le meta-apprenant. **Gauche** : Coefficients bruts (log-odds). Les coefficients positifs (vert) augmentent la probabilite de CKD, les negatifs (rouge) la diminuent. **Droite** : Importance relative en pourcentage. XGBoost et Random Forest sont les modeles les plus influents. 28
5.2 Decomposition de la prediction du Stacking pour un patient. **Haut gauche** : Predictions des modeles de base. **Haut droite** : Contributions ponderees par les coefficients. **Bas gauche** : Agregation cumulative. **Bas droite** : Comparaison avec la verite terrain. Cette decomposition permet au clinicien de suivre exactement le raisonnement du modele. 29
5.3 Importance des variables cliniques selon les valeurs SHAP pour chaque modele de base. Chaque sous-graphique montre les 10 variables les plus influentes. L'hemoglobine domine systematiquement, confirmant son role de biomarqueur critique de la CKD. La coherence entre modeles (arborescents et probabilistes) valide la robustesse des insights. 32

5.4 Comparaison multi-dimensionnelle entre Stacking conventionnel et XStacking. **Haut gauche** : Metriques de performance (Accuracy identique). **Haut droite** : Niveau d'explicabilite. **Bas gauche** : Complexite du modele (dimensionalite). **Bas droite** : Radar chart synthetisant les dimensions. XStacking excelle en explicabilite tout en maintenant les performances. 35

Liste des tableaux

1.1 Vue synthétique des cinq articles sur la prédiction de la MRC. 7
4.1 Statistiques descriptives des valeurs SHAP — Top 5 variables (forêt aléatoire). . 20
4.2 Importance des variables dans l'arbre distillé (Gini). 21
4.3 Statistiques descriptives des valeurs SHAP — Top 5 variables (XGBoost). 21
4.4 Moyenne et écart-type des valeurs SHAP de l'hémoglobine selon le statut diabétique (jeu de test, XGBoost). 22
4.5 Tableau de gouvernance — comparaison des propriétés d'explicabilité. 24
5.1 Comparaison detaillee Stacking vs XStacking 36

1 État de l'art

Cette section présente cinq articles consacrés à la prédiction de la maladie rénale chronique par apprentissage automatique. Chaque article est résumé ci-dessous ; le tableau 1.1 en donne une vue synthétique.

Almansour et al. (2022) [1]. Les auteurs comparent la forêt aléatoire, la machine à vecteurs de support et l'arbre de décision pour la prédiction et le staging de la maladie rénale chronique. La sélection de variables repose sur l'analyse de variance et l'élimination récursive des variables avec validation croisée. La forêt aléatoire avec élimination récursive surpasse la machine à vecteurs de support et l'arbre. L'étude souligne l'importance d'une détection précoce des stades de maladie rénale chronique pour limiter les complications (hypertension, anémie, troubles minéraux, etc.). Publication dans *Journal of Big Data*.

Swapna et Chandra (2021) [16]. Sur le jeu UCI maladie rénale chronique (400 observations, 25 paramètres), les auteurs combinent la forêt aléatoire et AdaBoost via un classifieur à vote. Le prétraitement inclut l'imputation des valeurs manquantes (en tenant compte de l'asymétrie) et une sélection de variables par corrélation. Le modèle hybride atteint jusqu'à 100 % d'exactitude selon les sous-ensembles de variables, avec validation sur plusieurs métriques (exactitude, précision, rappel, F1). Publication dans *EAI Endorsed Transactions on Pervasive Health and Technology*.

Swain et al. (2023) [15]. Les auteurs construisent un classifieur de maladie rénale chronique à partir de jeux publics en appliquant une imputation des manquants, un équilibrage des classes par surotéchantillonnage et une mise à l'échelle, puis une sélection de variables par test du Chi-carré. La machine à vecteurs de support et la forêt aléatoire sont comparées en validation croisée 10 folds : la machine à vecteurs de support obtient la meilleure exactitude (99,33 %), devant la forêt aléatoire (98,67 %). L'objectif est de réduire l'incertitude clinique en automatisant l'analyse des marqueurs. Publication dans *Electronics*.

Singh et al. (2024) [14]. Sur 491 patients (56 avec maladie rénale chronique, 435 sans), cinq modèles sont évalués (régression logistique, forêt aléatoire, arbre, Naïve Bayes, XGBoost) et expliqués par les méthodes SHAP et LIME. XGBoost obtient la meilleure performance (aire sous la courbe ROC 0,97, exactitude 93,29 %). Les variables les plus influentes sont la créatinine, l'hémoglobine glyquée et l'âge. L'étude montre que SHAP et LIME améliorent l'interprétabilité pour les cliniciens et favorisent une prédiction précoce de la maladie rénale chronique. Publication dans *Scientific Reports*.

Gogoi et al. (2024) [6]. Sur le jeu UCI maladie rénale chronique (400 exemples, 24 attributs), les auteurs utilisent une imputation par k plus proches voisins et une sélection de variables par algorithme génétique, puis comparent la forêt aléatoire, l'arbre de décision, la régression logistique et XGBoost. XGBoost atteint 99,17 % d'exactitude ; après sélection par algorithme génétique,

la forêt aléatoire et la régression logistique atteignent aussi 99,17 %. L'analyse SHAP identifie quatre variables dominantes : créatinine sérique, hémoglobine, gravité spécifique, albumine. Le système associe performance et interprétabilité pour la décision clinique. Publication dans *Discover Artificial Intelligence* (Springer).

TABLE 1.1 – Vue synthétique des cinq articles sur la prédiction de la maladie rénale chronique.

Référence	**Année**	**Données**	**Méthodes**	**Performance**
Almansour et al. [1]	2022	Maladie rénale chronique, binaire et stades	Forêt aléatoire, machine à vecteurs de support, arbre ; élimination récursive	Forêt aléatoire meilleure
Swapna & Chandra [16]	2021	UCI, 400 observations, 25 variables	Forêt aléatoire + AdaBoost, vote	Jusqu'à 100 %
Swain et al. [15]	2023	Maladie rénale chronique, jeu public	Surotéchantillonnage, Chi-carré ; machine à vecteurs de support, forêt aléatoire	99,33 % et 98,67 %
Singh et al. [14]	2024	491 patients	Régression logistique, forêt aléatoire, arbre, Naïve Bayes, XGBoost ; SHAP, LIME	XGBoost aire 0,97, 93 %
Gogoi et al. [6]	2024	UCI, 400 exemples, 24 variables	k plus proches voisins, algorithme génétique ; forêt aléatoire, arbre, régression logistique, XGBoost ; SHAP	XGBoost 99,17 %, 4 variables clés

1.1 Positionnement de notre étude

Ces travaux convergent sur l'importance du prétraitement (imputation par k plus proches voisins ou similaire, surotéchantillonnage, sélection de variables) et sur des performances élevées (exactitude typiquement >95 %, souvent 98–99 %) pour la forêt aléatoire, la machine à vecteurs de support, XGBoost et la régression logistique. Les variables les plus prédictives (créatinine, hémoglobine, gravité spécifique, albumine) sont cohérentes avec les marqueurs cliniques de la maladie rénale chronique. Les articles récents intègrent l'explicabilité (SHAP, LIME) pour rendre les prédictions utilisables en clinique. Notre rapport s'inscrit dans cette lignée en combinant un prétraitement reproductible sur le jeu UCI (k plus proches voisins, mode, LabelEncoder, surotéchantillonnage, StandardScaler), des modèles intrinsèquement interprétables (régression logistique, arbre), une machine à vecteurs de support expliquée par LIME et SHAP, et l'explicabilité des modèles d'ensemble via TreeSHAP (cas 3).

2 Prétraitement et visualisation des données

Cette section décrit la chaîne de prétraitement et d'exploration du jeu de données *Chronic Kidney Disease* (CKD) du dépôt UCI. L'objectif est de préparer les données pour les modèles d'explicabilité présentés aux sections suivantes : nettoyage, gestion des manquants, encodage, visualisation, puis découpage et équilibrage des classes. Toutes les étapes décrites correspondent à l'implémentation du notebook associé.

2.1 Chargement et nettoyage

Le jeu `kidney_disease.csv` contient 400 observations et 25 attributs (après suppression de l'identifiant). La cible est binaire : présence ou absence de maladie rénale chronique (CKD). La structure comprend :

— **Variables numériques** (14) : âge, tension artérielle (*blood_ pressure*), gravité spécifique (*specific_ gravity*), albumine, sucre, glucose sanguin, urée, créatinine sérique, sodium, potassium, hémoglobine, volume cellulaire, globules blancs/rouges, etc.

— **Variables catégorielles** (10) : globules rouges, cellules de pus, bactéries, hypertension, diabète, maladie coronarienne, appétit, œdème, anémie, etc.

Les colonnes ont été renommées de façon explicite (ex. `bp` → `blood_pressure`). Certaines colonnes initialement en format texte ont été converties en numérique (*packed_ cell_ volume*, *white_ blood_ cell_ count*, *red_ blood_ cell_ count*). Les valeurs catégorielles erronées (espaces, tabulations) ont été normalisées ; la cible a été encodée en binaire (0 = CKD, 1 = non CKD), et les lignes sans étiquette valide ont été supprimées (`dropna(subset=['class'])`).

```
df = pd.read_csv('kidney_disease.csv')
df.drop('id', axis=1, inplace=True)
df.columns = ['age', 'blood_pressure', 'specific_gravity', ...]
df['packed_cell_volume'] = pd.to_numeric(df['packed_cell_volume'],
    errors='coerce')
df['white_blood_cell_count'] = pd.to_numeric(df['white_blood_cell_count'
    ], errors='coerce')
df['red_blood_cell_count'] = pd.to_numeric(df['red_blood_cell_count'],
    errors='coerce')
df['class'] = df['class'].map({'ckd': 0, 'not␣ckd': 1})
df = df.dropna(subset=['class'])
```

2.2 Gestion des valeurs manquantes et encodage

Les valeurs manquantes ont été traitées de la manière suivante :

— **Variables numériques** : imputation par **KNN** (k plus proches voisins) avec $k = 5$ voisins (`KNNImputer` de scikit-learn), afin de préserver la structure locale des données.

— **Variables catégorielles** : remplacement des manquants par le **mode** de chaque colonne.

```
knn_imputer = KNNImputer(n_neighbors=5)
df[num_cols] = knn_imputer.fit_transform(df[num_cols])
for col in cat_cols:
    mode_val = df[col].mode()[0]
```

```
    df[col] = df[col].fillna(mode_val)
```

Ensuite, toutes les variables catégorielles ont été encodées numériquement à l'aide d'un **LabelEncoder** (une valeur entière par modalité), ce qui permet d'obtenir une matrice entièrement numérique pour la corrélation et les modèles.

```
le = LabelEncoder()
for col in cat_cols:
    df[col] = le.fit_transform(df[col].astype(str))
X = df.drop(columns=['class'])
y = df['class']
feature_names = X.columns.tolist()
```

2.3 Visualisation des données

La figure 2.1 présente les distributions des variables numériques (histogrammes). On observe des échelles et des formes variées (âge, pression, créatinine, hémoglobine, etc.), ce qui justifie l'utilisation d'un *StandardScaler* en aval pour les algorithmes sensibles à l'échelle.

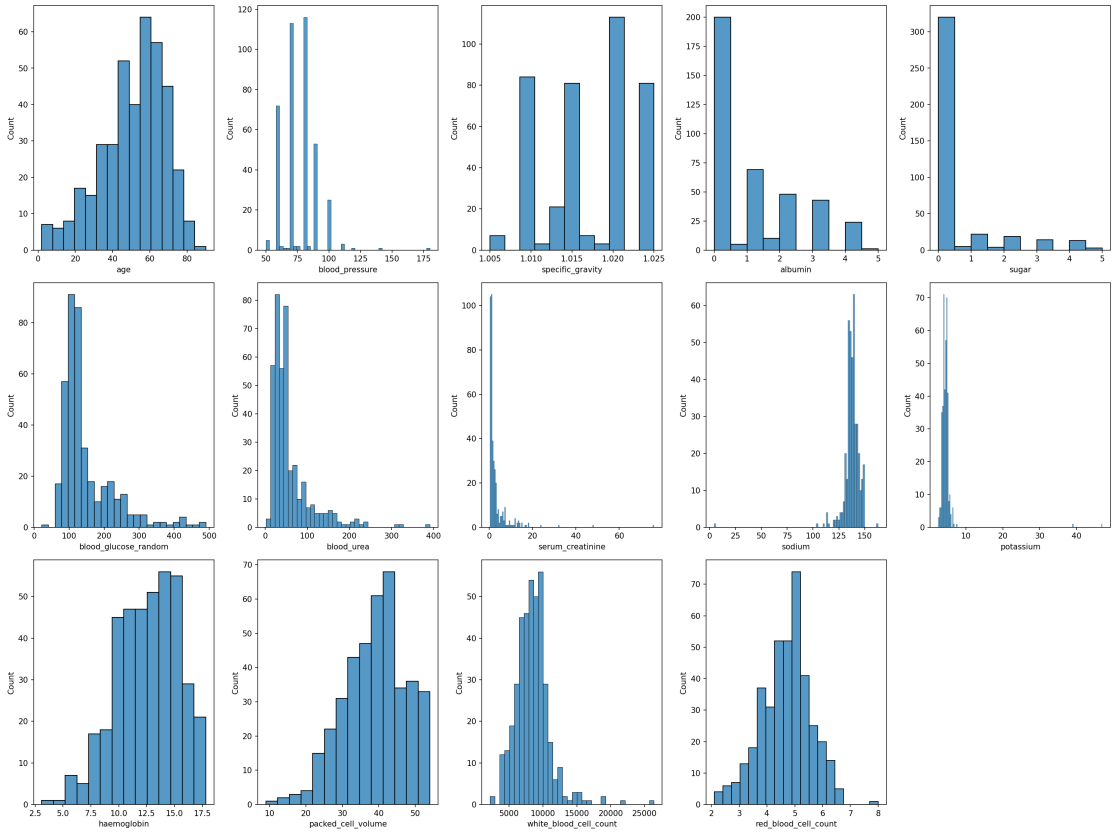

FIGURE 2.1 – Distributions des variables numériques du jeu CKD (après imputation).

La figure 2.2 montre les effectifs des modalités pour les variables catégorielles (avant encodage). Cela permet de vérifier l'équilibre relatif des catégories et d'identifier d'éventuelles modalités rares.

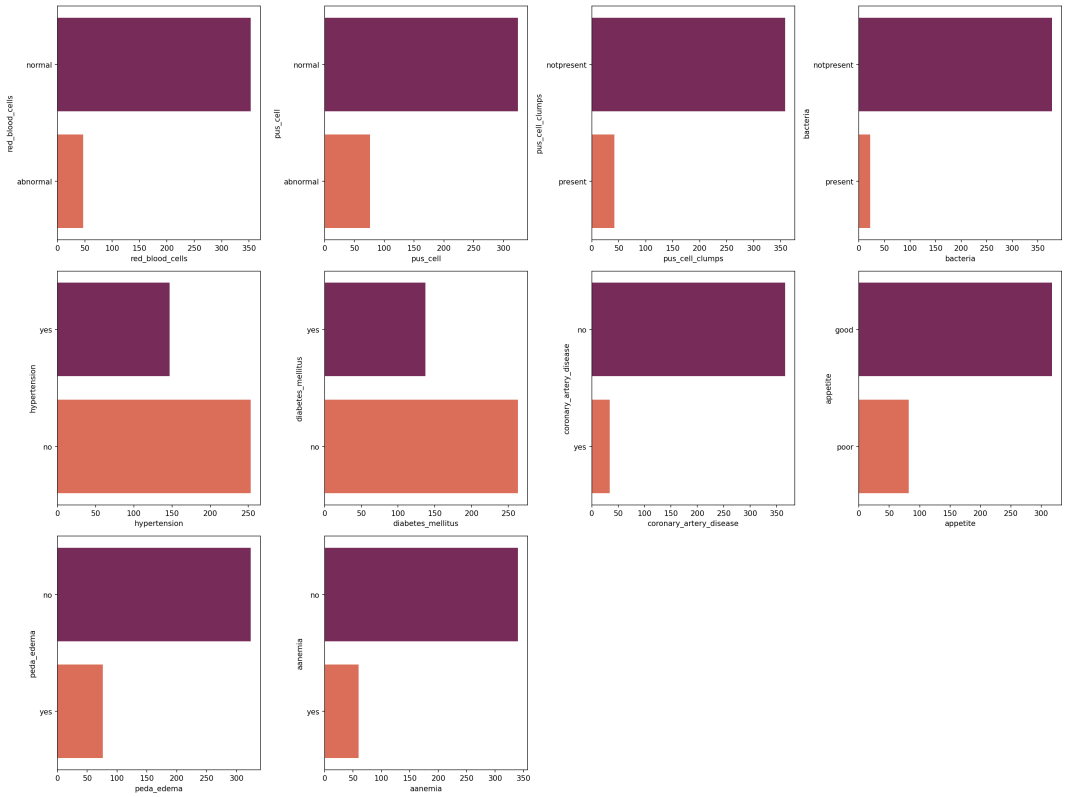

FIGURE 2.2 – Effectifs des variables catégorielles (avant encodage).

La figure 2.3 présente la matrice de corrélation. Les variables déjà encodées sont incluses ; on peut repérer les paires d'attributs fortement corrélés (ex. hémoglobine et volume cellulaire), utiles pour l'interprétation des modèles.

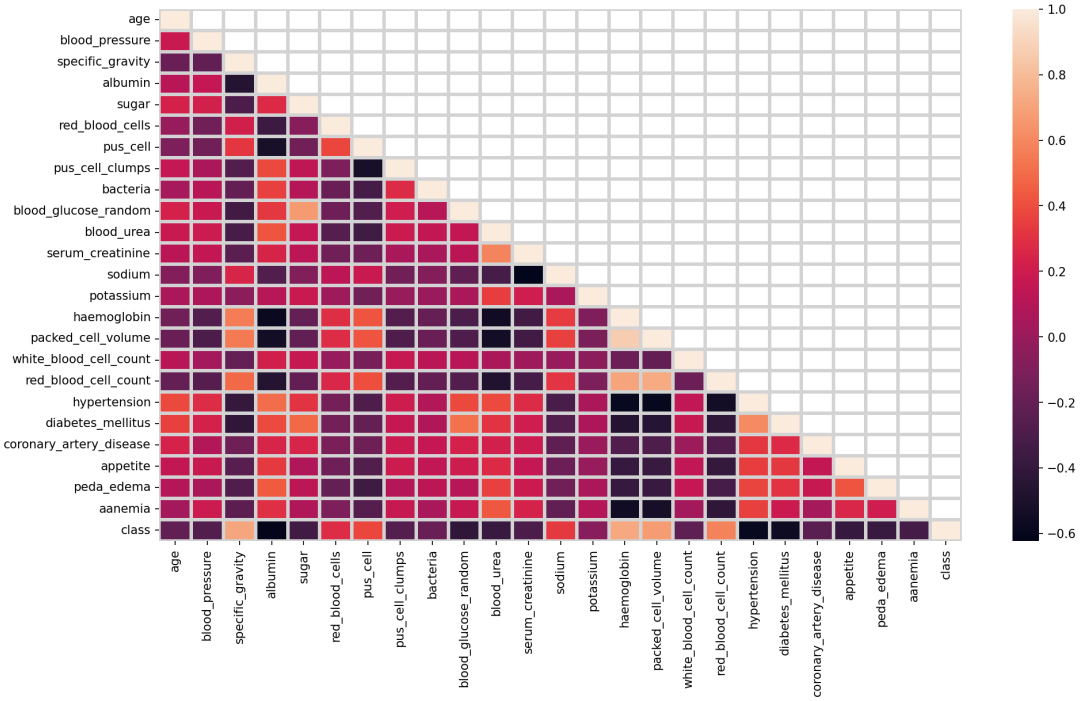

FIGURE 2.3 – Matrice de corrélation (triangle inférieur) entre toutes les variables du jeu CKD.

2.4 Découpage et équilibrage

Le jeu préparé est divisé en ensembles d'entraînement et de test par **découpage stratifié** (% 70 / 30) pour conserver les proportions de classes. La classe CKD étant souvent déséquilibrée, un rééquilibrage est appliqué sur l'ensemble d'entraînement uniquement avec **SMOTE** (technique de surotéchantillonnage de la minorité synthétique), avec $k = 3$ voisins, afin d'obtenir des effectifs de classes plus proches. Enfin, les variables numériques sont mises à l'échelle avec un **StandardScaler** (moyenne 0, écart-type 1), ajusté sur l'ensemble d'entraînement (après SMOTE) et appliqué aux données d'entraînement et de test. Les matrices ainsi obtenues (`X_train_scaled`, `X_test_scaled`) et les étiquettes associées sont utilisées dans les sections suivantes pour la régression logistique, l'arbre de décision, le SVM, LIME et SHAP.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=RANDOM_STATE)
smote = SMOTE(random_state=RANDOM_STATE, k_neighbors=3)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
scaler = StandardScaler()
X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train_smote),
   columns=feature_names)
X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=
   feature_names, index=X_test.index)
```

3 Interprétabilité intrinsèque des modèles et explicabilité post-hoc

Les données utilisées sont celles préparées à la section précédente (prétraitement et visualisation). On vise à illustrer deux approches complémentaires de l'explicabilité sur le jeu CKD : (1) l'**interprétabilité intrinsèque** via des modèles dont la structure fournit directement des explications (régression logistique, arbre de décision) ; (2) l'**explicabilité post-hoc** en expliquant a posteriori un modèle « boîte noire » (SVM à noyau) à l'aide de LIME et SHAP.

3.1 Interprétabilité intrinsèque

Régression logistique. Un modèle de régression logistique est entraîné sur les données SMOTE et standardisées. Les **coefficients** du modèle donnent une interprétation directe : chaque coefficient mesure l'effet d'une augmentation d'une unité de la variable (après standardisation) sur la tendance du modèle à prédire la classe CKD. Un coefficient positif indique un effet favorable au diagnostic CKD, un coefficient négatif un effet « protecteur ». La figure 3.1 présente ces coefficients triés par valeur absolue. D'après le notebook, les variables à fort effet positif (risque CKD) incluent notamment l'hypertension, le diabète, l'appétit, l'hémoglobine et la gravité spécifique ; à l'inverse, certaines variables (ex. certaines mesures de cellules ou d'œdème) ont un effet négatif.

```
lr = LogisticRegression(max_iter=2000, random_state=RANDOM_STATE,
    class_weight="balanced")
lr.fit(X_train_scaled, y_train_smote)
y_pred_lr = lr.predict(X_test_scaled)
```

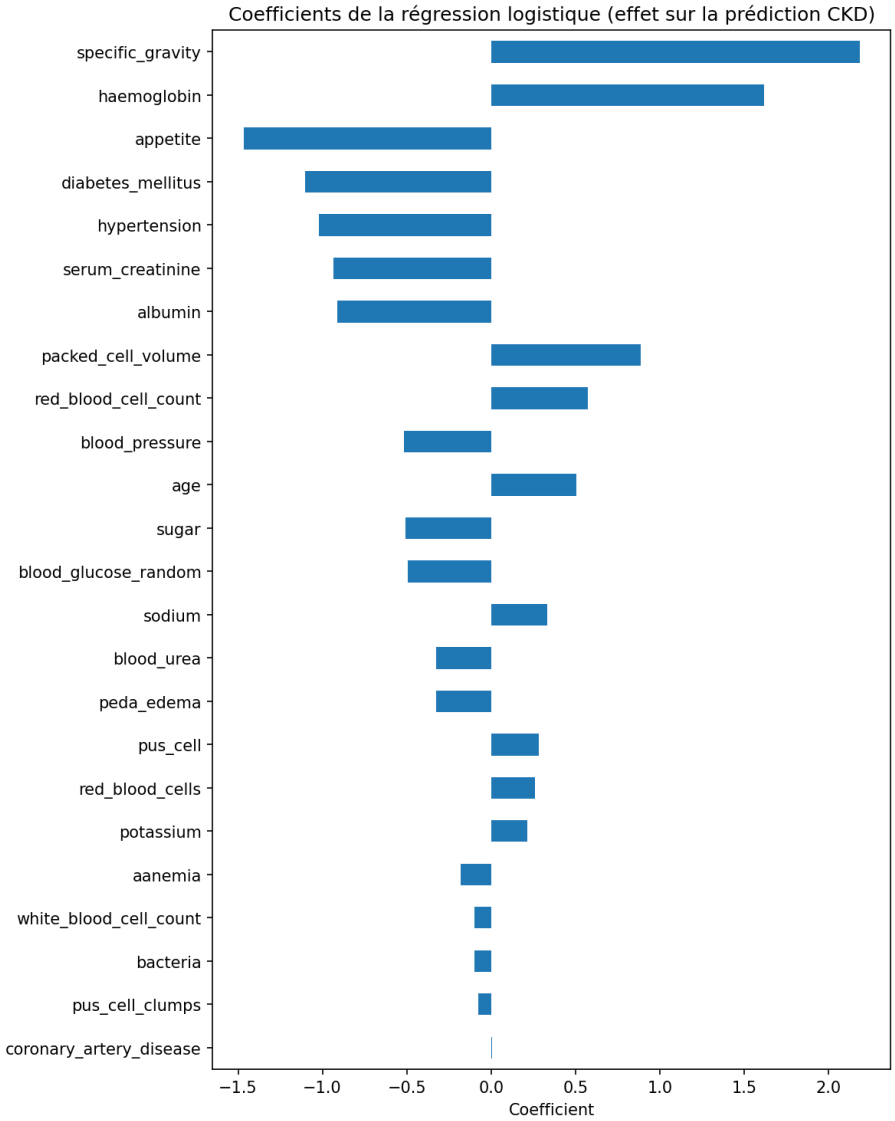

FIGURE 3.1 – Coefficients de la régression logistique (effet sur la prédiction CKD).

Arbre de décision. Un arbre de décision est entraîné avec une profondeur maximale de 4 (`max_depth=4`) pour rester lisible et éviter le surapprentissage. Chaque chemin de la racine vers une feuille se lit comme une règle « SI ... ALORS » : par exemple, « SI *specific_ gravity* $\leq$ seuil ET *haemoglobin* $\leq$ seuil ALORS forte probabilité CKD ». La figure 3.2 montre la structure complète de l'arbre (nœuds remplis, étiquettes de classes `ckd` / `not ckd`). L'importance des variables (basée sur l'impureté) met en avant, entre autres, *specific_ gravity*, *haemoglobin* et l'albumine, cohérent avec la littérature sur les marqueurs rénaux.

```
dt = DecisionTreeClassifier(max_depth=4, random_state=RANDOM_STATE)
dt.fit(X_train_scaled, y_train_smote)
y_pred_dt = dt.predict(X_test_scaled)
```

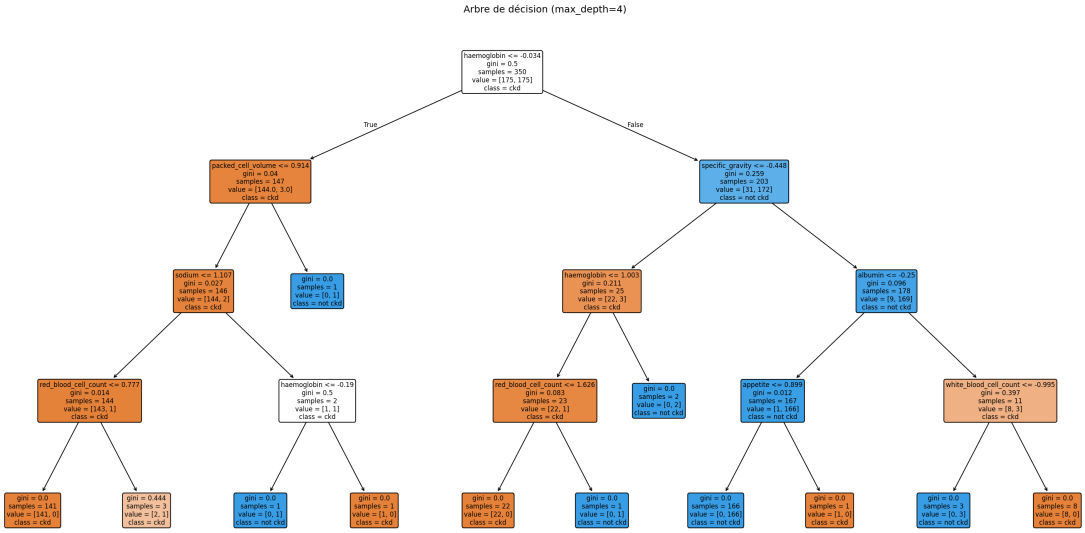

FIGURE 3.2 – Arbre de décision (max_depth=4) pour la prédiction CKD.

3.2 Explicabilité post-hoc des modèles opaques

Un **SVM à noyau RBF** est utilisé comme modèle « boîte noire » : il n'offre pas d'interprétation directe des prédictions. L'option `probability=True` est activée pour exposer `predict_proba`, nécessaire à LIME et SHAP. Les performances (exactitude, AUC, matrice de confusion) sont évaluées sur l'ensemble de test ; les valeurs exactes sont reportées dans le notebook. Le modèle sert ensuite de cible pour les explicateurs locaux et globaux.

```
svm = SVC(kernel="rbf", probability=True, random_state=RANDOM_STATE)
svm.fit(X_train_scaled, y_train_smote)
y_pred_svm = svm.predict(X_test_scaled)
```

LIME (explications locales interprétables, agnostiques au modèle). LIME approxime localement le comportement du SVM par un modèle interprétable (ex. régression linéaire) autour d'une observation donnée. On perturbe cette observation, on récupère les prédictions du SVM, puis on pondère les points par leur proximité et on ajuste un modèle linéaire dont les coefficients sont les « poids LIME » : ils indiquent la contribution de chaque variable à la prédiction pour cette observation. La figure 3.3 montre ces poids pour une observation de test (patient) ; les barres vertes (positives) et rouges (négatives) indiquent quelles variables poussent vers ou contre la classe CKD. Une limite connue de LIME est l'instabilité des explications d'une exécution à l'autre (variabilité de l'échantillonnage).

```
def predict_proba_ckd(X_arr):
    return svm.predict_proba(X_arr)[:, 0]
explainer_lime = lime.lime_tabular.LimeTabularExplainer(
    X_train_scaled.values, feature_names=feature_names,
    class_names=["ckd", "not␣ckd"], mode="regression", random_state=
        RANDOM_STATE)
exp = explainer_lime.explain_instance(instance, predict_proba_ckd,
   num_features=10)
```

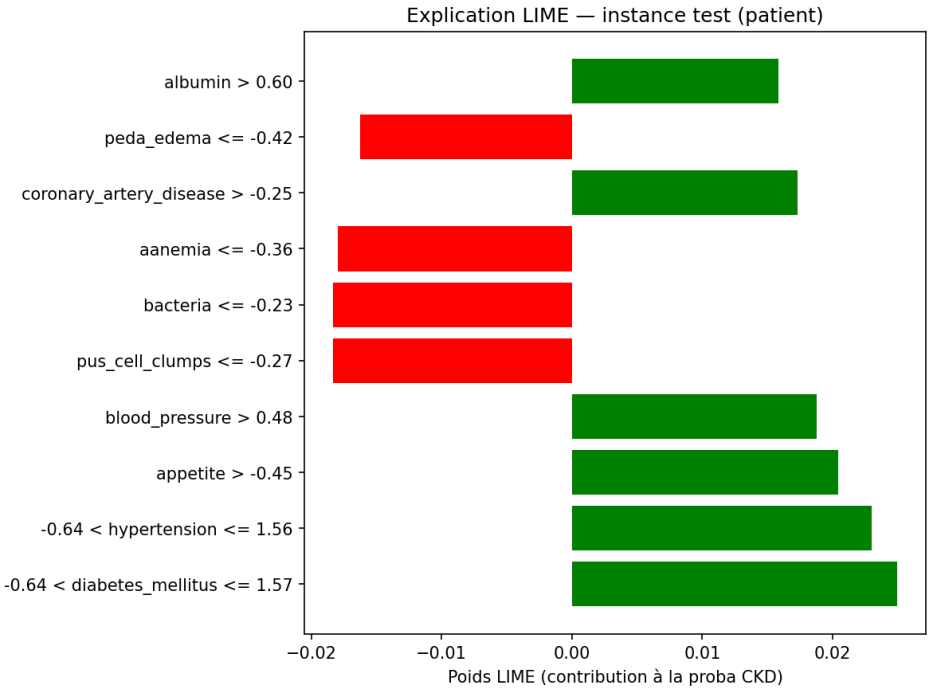

FIGURE 3.3 – Poids LIME pour une observation de test : contribution de chaque variable à la probabilité CKD.

SHAP (explications additives de Shapley). SHAP repose sur les **valeurs de Shapley** de la théorie des jeux : chaque variable reçoit une contribution (valeur SHAP) à la prédiction, avec la propriété d'additivité (somme des SHAP = écart entre la prédiction et la moyenne). Un explicateur à noyau (**KernelExplainer**) est utilisé avec un jeu de fond (échantillon d'entraînement) et une fonction de prédiction donnant la probabilité de la classe CKD. La figure 3.4 présente les valeurs SHAP pour la même observation que LIME (explication locale) ; la figure 3.5 est un graphique récapitulatif global sur un échantillon du jeu de test : chaque point correspond à une observation, la couleur indique la valeur de la variable et l'axe horizontal la valeur SHAP. On peut ainsi identifier les variables les plus influentes globalement et le sens de leur effet (hautes valeurs en rouge, basses en bleu). Les explications SHAP sont en général plus stables que LIME et offrent une vue à la fois locale (par observation) et globale (résumé sur l'échantillon).

```
def predict_proba_ckd_shap(X):
    return svm.predict_proba(X)[:, 0]
background = X_train_scaled.sample(50, random_state=RANDOM_STATE)
explainer_shap = shap.KernelExplainer(predict_proba_ckd_shap, background
    , link="logit")
shap_values_one = explainer_shap.shap_values(instance, nsamples=100)
```

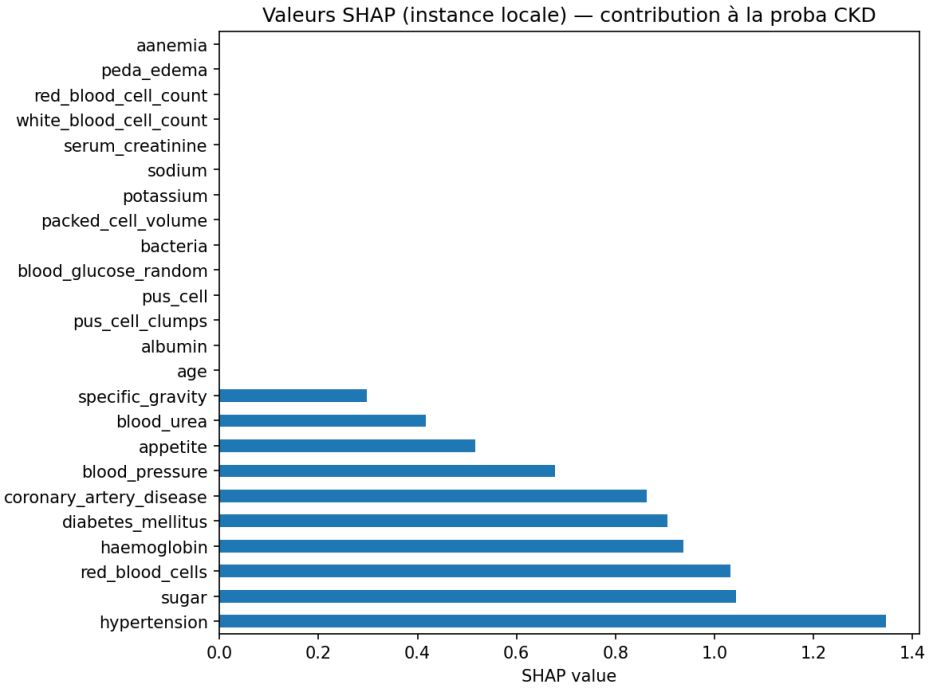

FIGURE 3.4 – Valeurs SHAP (explication locale) pour une observation de test.

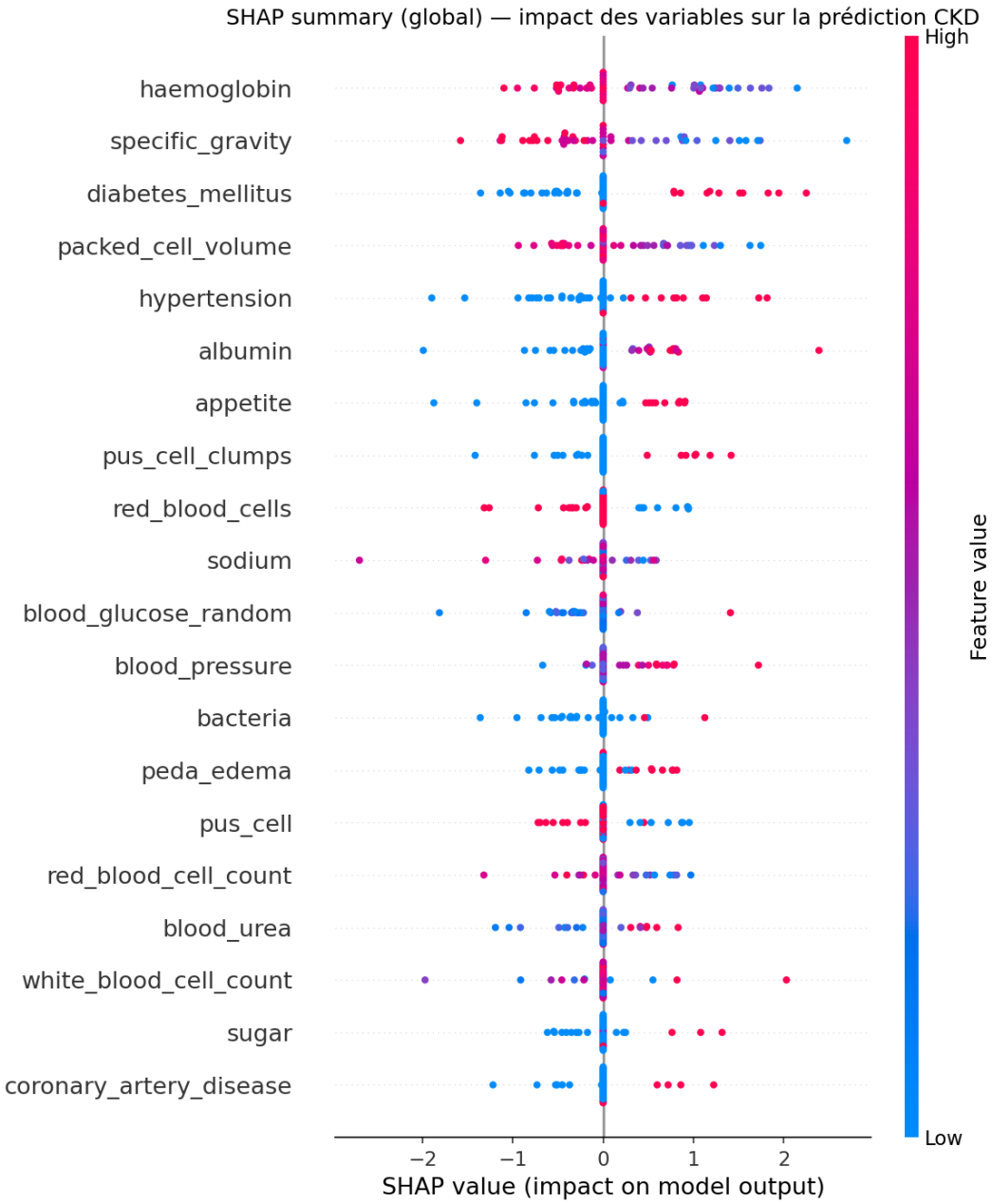

FIGURE 3.5 – Résumé SHAP (vue globale) : impact des variables sur la prédiction CKD.

3.3 Synthèse

Les modèles intrinsèquement interprétables (régression logistique, arbre de décision) fournissent directement des règles et des coefficients exploitables pour comprendre les prédictions. Pour un modèle opaque comme le SVM, les méthodes post-hoc LIME et SHAP permettent d'expliquer des prédictions individuelles (LIME, SHAP local) et de résumer l'importance des variables à l'échelle du jeu (SHAP global). La combinaison des deux approches (modèles interprétables pour la transparence et méthodes post-hoc pour les modèles complexes) couvre un large besoin en explicabilité dans le cadre de la prédiction de la maladie rénale chronique.

4 Explicabilité des modèles de combinaison (bagging et boosting)

Les données utilisées sont celles préparées aux sections précédentes (jeu UCI maladie rénale chronique, CKD). On vise à illustrer l'explicabilité de deux modèles d'ensemble : une **forêt aléatoire** (*Random Forest*, RF) et un modèle à **gradient boosting** (*XGBoost*, XGB). Plusieurs techniques complémentaires sont appliquées — TreeSHAP, distillation de connaissances, analyse des interactions SHAP — pour fournir une vue locale, globale et auditables des prédictions.

4.1 Modèles construits

Deux modèles sont entraînés sur un jeu d'entraînement représentant 80 % des données (stratifié sur la variable cible), le reste étant réservé à l'évaluation et à l'explicabilité.

```
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

xgb = XGBClassifier(n_estimators=100, use_label_encoder=False,
                    eval_metric='logloss', random_state=42)
xgb.fit(X_train, y_train)
```

Les deux modèles obtiennent des performances très élevées sur le jeu de test (exactitude supérieure à 98 %), cohérent avec la littérature. L'objet ici n'est pas de comparer les performances mais d'analyser la **transparence** de ces modèles au moyen de techniques d'explicabilité.

4.2 TreeSHAP appliqué à la forêt aléatoire

Principe et mise en œuvre. TreeSHAP [8] est une implémentation efficace des valeurs de Shapley pour les modèles arborescents : elle permet de calculer exactement, en temps polynomial, la contribution marginale de chaque variable à chaque prédiction. Un explicateur `TreeExplainer` est construit sur la forêt ; les valeurs SHAP sont calculées sur l'ensemble de test. La figure 4.1 présente le résumé agrégé (beeswarm) : chaque point est une observation, la couleur encode la valeur de la variable (rouge = élevée, bleu = faible), l'axe horizontal la contribution SHAP à la prédiction.

```
import shap

explainer_rf = shap.TreeExplainer(rf)
shap_values_rf = explainer_rf.shap_values(X_test)

shap.summary_plot(shap_values_rf[1], X_test,
                  feature_names=feature_names, show=False)
plt.savefig('shap_rf_summary.png', bbox_inches='tight', dpi=150)
```

```
plt.close()
```

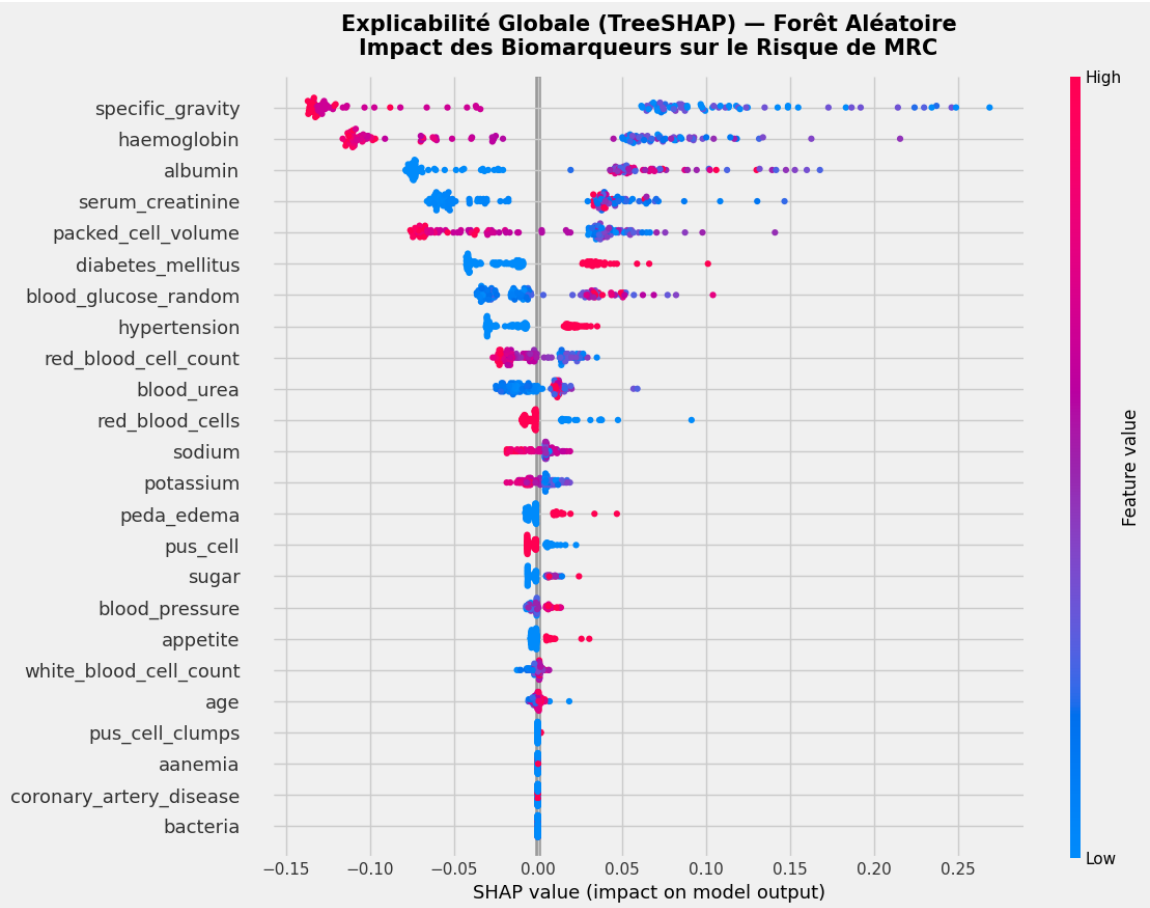

FIGURE 4.1 – Valeurs SHAP agrégées sur le jeu de test — forêt aléatoire.

Interprétation. Le classement d'importance globale (Mean |SHAP|) place en tête `specific_gravity` (22 %), `haemoglobin` (16 %), `albumin` (12,9 %), `serum_creatinine` (9,5 %) et `packed_cell_volume` (9,2 %). Les corrélations de Spearman entre la valeur de chaque variable et sa valeur SHAP indiquent le sens de l'effet : négatif signifie que plus la variable augmente, plus le risque prédit baisse (ex. `haemoglobin` $r = -0{,}81$, `specific_gravity` $r = -0{,}81$, `packed_cell_volume` $r = -0{,}80$) ; positif signifie que plus la variable augmente, plus le risque prédit augmente (ex. `serum_creatinine` $r = +0{,}76$, `albumin` $r = +0{,}79$, `hypertension` $r = +0{,}82$). Un taux d'hémoglobine faible contribue ainsi positivement au risque CKD (anémie associée à l'insuffisance rénale) ; une créatinine sérique élevée augmente le risque prédit (réduction du débit de filtration glomérulaire). Le tableau 4.1 résume les statistiques descriptives des valeurs SHAP pour les cinq variables les plus importantes.

TABLE 4.1 – Statistiques descriptives des valeurs SHAP — Top 5 variables (forêt aléatoire).

Stat.	`specific_gravity`	`haemoglobin`	`albumin`	`serum_creat.`	`packed_cell_vol.`
count	120	120	120	120	120
mean	0,014	−0,007	0,006	0,003	−0,001
std	0,124	0,089	0,073	0,053	0,053
min	−0,137	−0,116	−0,079	−0,066	−0,076
max	0,269	0,216	0,168	0,147	0,141

4.3 Distillation de connaissances

Principe. La distillation consiste à entraîner un modèle simple et interprétable (ici un arbre de décision de profondeur 4) à reproduire les prédictions probabilistes (*soft labels*) de la forêt aléatoire. On obtient une approximation globale du modèle opaque sous la forme d'un arbre lisible. Chaque feuille donne une probabilité CKD ; le seuil de décision clinique est 0,5. La figure 4.2 montre la structure de l'arbre distillé.

```
from sklearn.tree import DecisionTreeClassifier, export_text

soft_labels = rf.predict_proba(X_train)[:, 1]
student_tree = DecisionTreeClassifier(max_depth=4, random_state=42)
student_tree.fit(X_train, (soft_labels >= 0.5).astype(int))
tree_rules_text = export_text(student_tree, feature_names=list(X_train.
    columns))
```

La structure de l'arbre (pseudo-code clinique) est la suivante. La racine sépare sur `specific_gravity` $\leq 1{,}02$; les nœuds suivants font intervenir `haemoglobin`, `packed_cell_volume`, `albumin`, `diabetes_mellitus`, `hypertension`, `blood_glucose_random` et `sodium`. Les feuilles donnent des valeurs (probabilités) entre 0,02 et 1,00 ; une probabilité $\geq 0{,}5$ conduit à la prédiction CKD.

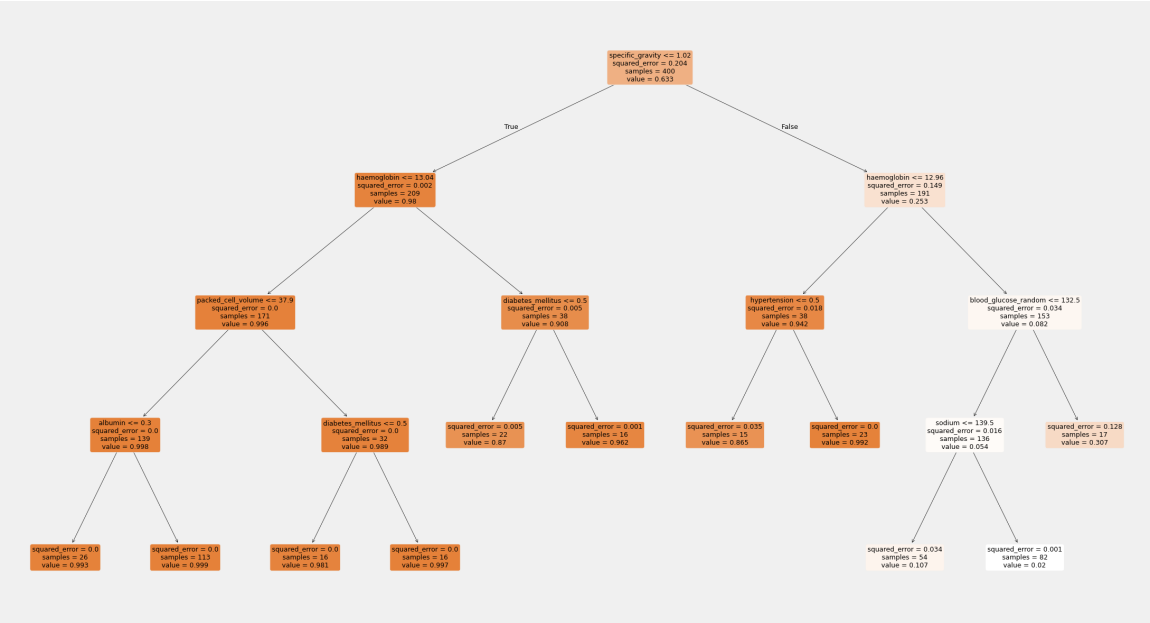

FIGURE 4.2 – Arbre de décision distillé depuis la forêt aléatoire (profondeur max. 4).

Importance des variables (Gini) et interprétation. L'importance des variables dans l'arbre distillé (impureté de Gini) est dominée par `specific_gravity` (68,6 %) et `haemoglobin` (29,6 %) ; viennent ensuite `blood_glucose_random`, `sodium`, `hypertension` et `diabetes_mellitus` (tableau 4.2). La fidélité de l'arbre distillé (accord avec les prédictions de la RF sur le jeu de test) est supérieure à 96 % : le comportement global du modèle opaque est bien capturé par un arbre de profondeur 4. Cette hiérarchie (gravité spécifique et hémoglobine en tête) est **cohérente avec les valeurs SHAP** (section 4.2).

TABLE 4.2 – Importance des variables dans l'arbre distillé (Gini).

Variable	**Importance (%)**
`specific_gravity`	68,6
`haemoglobin`	29,6
`blood_glucose_random`	1,3
`sodium`	0,3
`hypertension`	0,2
`diabetes_mellitus`	0,1

4.4 Interactions SHAP appliquées à XGBoost

Classement global et corrélations (XGBoost). Pour XGBoost, le classement d'importance globale (Mean |SHAP|) place en tête `haemoglobin` (19,3 %), `specific_gravity` (18,9 %), `serum_creatinine` (14 %), `albumin` (12,5 %) et `packed_cell_volume` (7,4 %). Les corrélations de Spearman (valeur de la variable ↔ SHAP) indiquent le sens de l'effet : `haemoglobin` $r = +0{,}81$, `specific_gravity` $r = +0{,}79$, `serum_creatinine` $r = -0{,}76$, `albumin` $r = -0{,}80$, `blood_glucose_random` $r = -0{,}90$. Le tableau 4.3 donne les statistiques descriptives des valeurs SHAP pour les cinq variables les plus importantes.

TABLE 4.3 – Statistiques descriptives des valeurs SHAP — Top 5 variables (XGBoost).

Stat.	`haemoglobin`	`specific_gravity`	`serum_creat.`	`albumin`	`packed_cell_vol.`
count	120	120	120	120	120
mean	−0,005	−0,534	−0,092	−0,356	−0,149
std	1,686	1,588	1,230	1,053	0,617
min	−3,23	−3,78	−2,15	−2,42	−1,37
max	2,39	1,59	1,71	0,85	0,90

Dependence plot et interaction Hémoglobine × Diabète. Les valeurs SHAP d'interaction généralisent les valeurs de Shapley au second ordre. Le graphique de dépendance représente, pour une variable donnée, sa contribution SHAP en fonction de sa valeur, avec une coloration par une seconde variable, ce qui révèle visuellement les interactions. La figure 4.3 illustre l'hémoglobine en fonction du statut diabétique.

```
import shap

explainer_xgb = shap.TreeExplainer(xgb)
shap_values_xgb = explainer_xgb.shap_values(X_test)
```

```
shap.dependence_plot(
    "haemoglobin", shap_values_xgb, X_test_df,
    interaction_index="diabetes_mellitus",
    feature_names=feature_names, show=False)
plt.savefig('shap_xgb_dependence_hemo_dm.png',
            bbox_inches='tight', dpi=150)
plt.close()
```

Le tableau 4.4 donne la moyenne et l'écart-type des valeurs SHAP de l'hémoglobine selon le statut diabétique (jeu de test, XGBoost). La corrélation de Spearman entre le taux d'hémoglobine et sa valeur SHAP est $r = +0{,}860$ dans le groupe non diabétique et $r = +0{,}069$ dans le groupe diabétique.

TABLE 4.4 – Moyenne et écart-type des valeurs SHAP de l'hémoglobine selon le statut diabétique (jeu de test, XGBoost).

Groupe	**Moyenne SHAP**	**Écart-type**	**Effectif**
Non diabétique (dm = 0)	+0.5669	1.6606	85
Diabétique (dm = 1)	−1.3931	0.5789	35

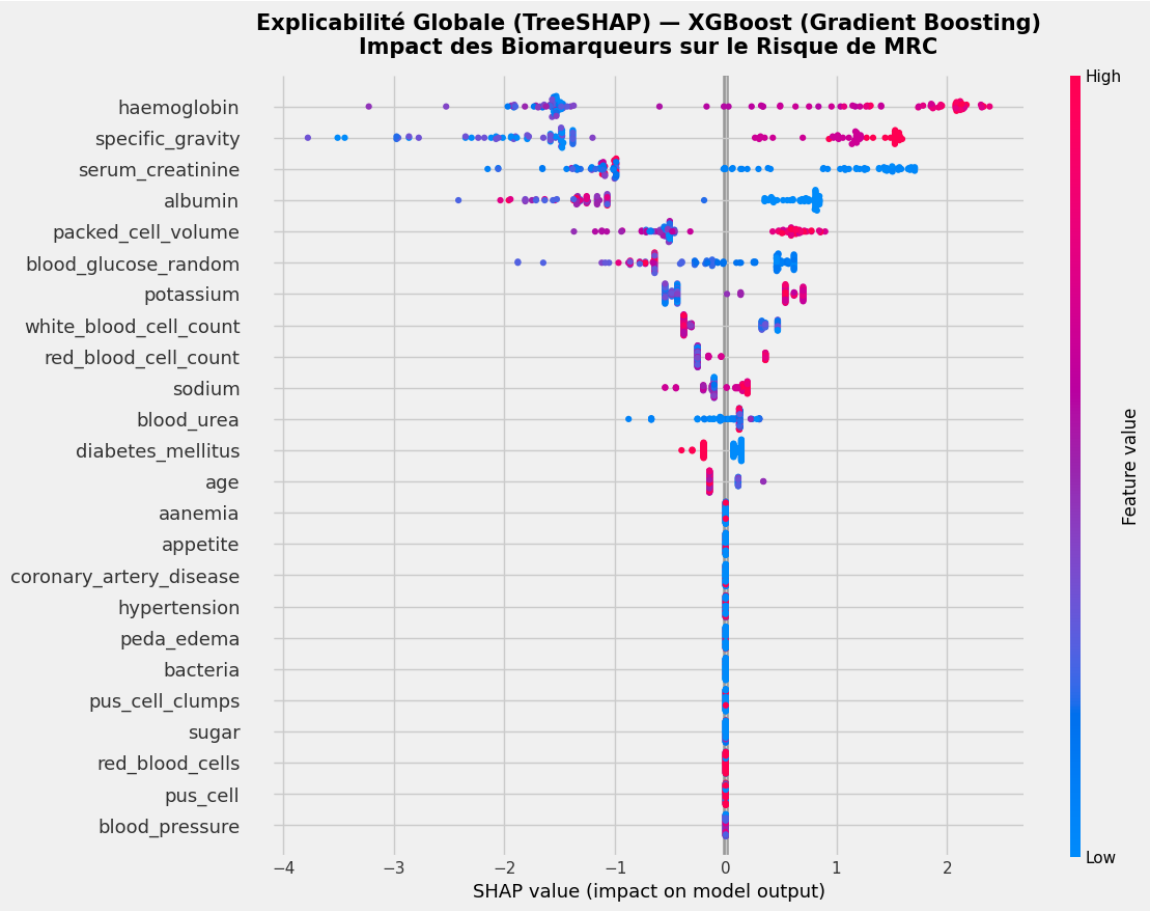

FIGURE 4.3 – Graphique de dépendance SHAP (XGBoost). Axe des abscisses : taux d'hémoglobine (g/dL). Couleur : rouge = diabétique (dm = 1), bleu = non diabétique (dm = 0).

Interprétation. La moyenne des valeurs SHAP pour l'hémoglobine est $+0{,}57$ chez les non diabétiques (dm $= 0$) et $-1{,}39$ chez les diabétiques (dm $= 1$) (tableau 4.4). La corrélation Hémoglobine $\leftrightarrow$ SHAP (hémoglobine) est $r = +0{,}86$ dans le groupe non diabétique et $r = +0{,}07$ dans le groupe diabétique : une corrélation négative signifierait que plus l'hémoglobine baisse, plus la contribution au risque CKD augmente ; ici, chez les diabétiques, l'hémoglobine perd sa valeur discriminante, le modèle s'appuyant davantage sur d'autres variables. Dans la plage intermédiaire (environ 10–12 g/dL), les points rouges (diabétiques) se situent au-dessus des points bleus : pour un même degré d'anémie modérée, un patient diabétique reçoit une contribution SHAP plus élevée (risque plus grand). Ceci reflète l'interaction biologique de la **néphropathie diabétique**. L'analyse SHAP permet ainsi de détecter des **interactions non linéaires** que les méthodes d'importance globale ne révèlent pas.

Valeurs d'interaction Shapley-Taylor et graphique clinique. Pour aller au-delà des contributions marginales, on calcule le **tenseur d'interactions SHAP** (décomposition de type Shapley-Taylor) via `shap_interaction_values` : pour chaque observation, on obtient une matrice $p \times p$ qui décompose les contributions au second ordre (interaction entre paires de variables). La forme du tenseur est (n, p, p). Le graphique de dépendance (hémoglobine en abscisse, valeur SHAP en ordonnée, coloration par `diabetes_mellitus` avec nuancier coolwarm) est ensuite produit à partir des valeurs SHAP standard ; un titre explicite met en avant l'« interaction clinique : Hémoglobine × Diabète sucré » et l'effet non linéaire et modulé de l'hémoglobine sur le risque de CKD. La figure 4.4 présente ce graphique.

```
shap_interaction_values = explainer_xgb.shap_interaction_values(X_test)

shap_vals_xgb = np.asarray(shap_values_xgb, dtype=np.float64)
X_test_vals = np.asarray(X_test, dtype=np.float64)
shap.dependence_plot(
    "haemoglobin", shap_vals_xgb, X_test_vals,
    interaction_index="diabetes_mellitus",
    feature_names=list(X_test.columns), show=False,
    cmap=plt.get_cmap("coolwarm"))
ax = plt.gca()
ax.set_title(
    "Interaction Clinique : Hemoglobine x Diabete Sucre\n"
    "Effet non lineaire et module (XGBoost)", fontsize=14)
ax.set_xlabel("Taux d'Hemoglobine (g/dL)", fontsize=12)
ax.set_ylabel("Valeur SHAP (contribution au log-odds CKD)", fontsize=11)
plt.tight_layout()
plt.savefig('output4.png', bbox_inches='tight', dpi=150)
```

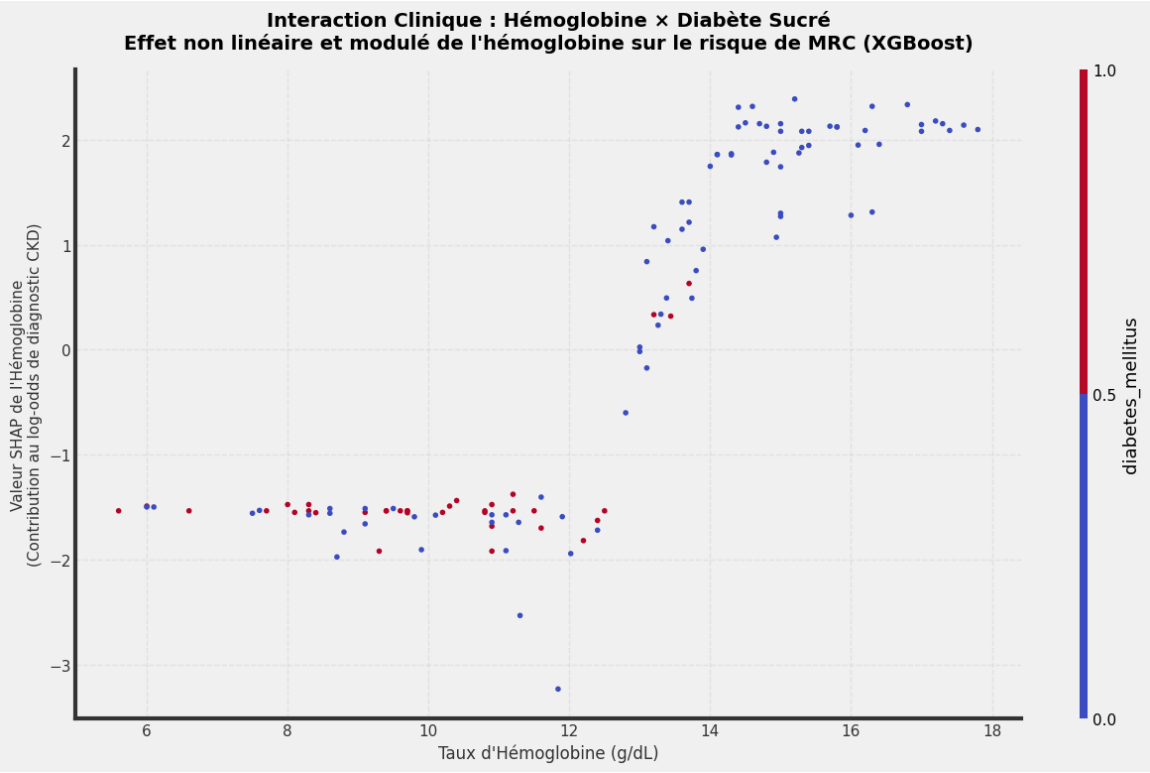

FIGURE 4.4 – Interaction clinique : Hémoglobine × Diabète sucré — effet non linéaire et modulé de l'hémoglobine sur le risque de CKD (XGBoost). Axe des abscisses : taux d'hémoglobine (g/dL). Couleur : `diabetes_mellitus` (bleu = faible, rouge = élevé).

Ce graphique résume visuellement comment le statut diabétique module la relation entre le taux d'hémoglobine et sa contribution au log-odds de diagnostic CKD : en zone intermédiaire (environ 12–14 g/dL), la pente et le niveau des valeurs SHAP diffèrent selon que le patient est diabétique ou non, ce qui illustre la **néphropathie diabétique** sans qu'elle soit encodée explicitement dans le modèle.

4.5 Tableau de gouvernance

En vue d'auditabilité et de conformité (ex. cadre de l'IA Act européen), les propriétés des deux modèles sont synthétisées dans le tableau 4.5.

TABLE 4.5 – Tableau de gouvernance — comparaison des propriétés d'explicabilité.

Critère	**Random Forest**	**XGBoost**
Explicabilité locale (SHAP)	Oui (TreeSHAP)	Oui (TreeSHAP)
Distillation possible	Oui (fidélité $> 96\,\%$)	Oui
Interactions détectables (SHAP)	Oui	Oui (plus stable)
Stabilité des explications	Modérée	Élevée
Complexité du modèle	Modérée	Élevée

XGBoost présente des valeurs SHAP plus stables grâce à la régularisation (L_1, L_2) intégrée.

4.6 Synthèse

Les techniques d'explicabilité appliquées aux modèles d'ensemble (RF, XGBoost) sur le jeu CKD donnent des résultats cohérents et cliniquement pertinents : la gravité spécifique (`specific_gravity`), l'hémoglobine, l'albumine, la créatinine sérique et le volume globulaire apparaissent comme les variables les plus contributives (SHAP global, importance Gini dans l'arbre distillé). La distillation valide qu'un arbre de profondeur 4 approxime bien le comportement de la forêt avec une fidélité élevée, la racine et l'importance Gini mettant en avant la gravité spécifique et l'hémoglobine. L'analyse d'interaction SHAP (moyenne SHAP par groupe diabète, corrélations $r = +0{,}86$ / $r = +0{,}07$) révèle que le diabète est un modificateur d'effet fort sur la relation hémoglobine–risque CKD, en accord avec la néphropathie diabétique. La combinaison d'explications locales (SHAP) et de modèles distillés couvre un large besoin en explicabilité pour les applications médicales.

5 Approches avancees d'explicabilite : Stacking et XStacking

Cette section explore des techniques d'ensemble learning avancees (Stacking, XStacking) en mettant l'accent sur leur explicabilite. Le Stacking combine les predictions de plusieurs modeles de base via un meta-apprenant, mais pose un defi majeur : **la perte de tracabilite**. Nous proposons le paradigme **XStacking** qui enrichit le Stacking avec des valeurs SHAP pour restaurer l'explicabilite end-to-end.

5.1 Stacking conventionnel avec validation croisee

Principe du Stacking

Le **Stacking** (ou *Stacked Generalization*, Wolpert 1992) est une methode d'ensemble learning a deux niveaux :

1. **Niveau 0 (modeles de base)** : Un ensemble de M modeles heterogenes $\{h_1, h_2, \ldots, h_M\}$ est entraine sur les donnees d'entrainement. Pour eviter le surapprentissage, on utilise la validation croisee stratifiee (K-Fold) pour generer des predictions *Out-Of-Fold* (OOF) : chaque prediction $\hat{y}_i^{(m)}$ est produite par un modele h_m qui n'a pas vu l'observation (x_i, y_i) lors de l'entrainement.
2. **Niveau 1 (meta-apprenant)** : Un meta-apprenant g est entraine sur un nouveau dataset constitue des predictions OOF : $\mathcal{D}_{meta} = \{([\hat{y}_i^{(1)}, \ldots, \hat{y}_i^{(M)}], y_i)\}_{i=1}^N$. Le meta-apprenant apprend a combiner optimalement les predictions des modeles de base.

La prediction finale pour une nouvelle observation x est :

$$\hat{y} = g(h_1(x), h_2(x), \ldots, h_M(x)) \tag{1}$$

Configuration des modeles de base

Le succes du Stacking repose sur la **diversite** des modeles de base. Nous avons selectionne 5 modeles couvrant differents paradigmes d'apprentissage pour maximiser la complementarite :

— **KNN** : Apprentissage par proximite locale, non-parametrique

— **Extra Trees** : Ensemble avec splits aleatoires (randomisation accrue)

— **Random Forest (130 arbres)** : Ensemble d'arbres par Bagging (reduction de variance)

— **XGBoost (150 arbres)** : Gradient Boosting optimise (correction des erreurs residuelles)

— **Naive Bayes** : Modele probabiliste (hypothese d'independance)

Cette diversite garantit que les modeles ont des biais et des variances differents, condition necessaire pour que le Stacking ameliore la performance (theoreme de Wolpert).

Listing 1 – Configuration des modeles de base avec principe de complementarite

```
from sklearn.ensemble import RandomForestClassifier,
   ExtraTreesClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from xgboost import XGBClassifier

base_learners = [
```

```
    ('knn', KNeighborsClassifier()),

    ('et', ExtraTreesClassifier()),

    ('rf', RandomForestClassifier(
        n_estimators=130,
        max_depth=11,
        min_samples_split=3,
        min_samples_leaf=2,
        max_features='sqrt',
        criterion='entropy',
        bootstrap=True,
        random_state=42,
        n_jobs=-1
    )),

    ('xgb', XGBClassifier(
        n_estimators=150,
        max_depth=5,
        learning_rate=0.5,
        objective='binary:logistic'
    )),

    ('nb', GaussianNB(var_smoothing=1e-9))
]
```

Construction du Stacking

Le meta-apprenant choisi est une **Regression Logistique** pour sa simplicite et son explicabilite intrinseque. Ses coefficients β_m indiquent directement l'importance de chaque modele de base dans la decision finale :

$$P(y = 1|\hat{\mathbf{y}}) = \sigma\left(\beta_0 + \sum_{m=1}^{M} \beta_m \hat{y}^{(m)}\right) \tag{2}$$

ou σ est la fonction sigmoide et $\hat{y}^{(m)}$ la probabilite predite par le modele m.

Listing 2 – Construction du Stacking avec validation croisee

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

meta_learner = LogisticRegression(
    penalty='l2',
    C=1.0,
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)
```

```
stacking_clf = StackingClassifier(
    estimators=base_learners,
    final_estimator=meta_learner,
    cv=6,
    n_jobs=-1,
    passthrough=False
)

stacking_clf.fit(X_train, y_train)
y_pred_stacking = stacking_clf.predict(X_test)
y_proba_stacking = stacking_clf.predict_proba(X_test)
```

Resultats

Le modele de Stacking atteint une **accuracy de 95.83%** sur l'ensemble de test, egalant la performance du meilleur modele individuel (Extra Trees) tout en beneficiant d'une robustesse accrue grace a la diversite des modeles de base.

Explicabilite du meta-apprenant

Les coefficients de la regression logistique revelent l'importance de chaque modele de base dans la decision finale. La figure 5.1 presente ces coefficients : on observe que **XGBoost** et **Random Forest** dominent avec les coefficients les plus eleves, tandis que **Naive Bayes** a un coefficient negatif, suggerant qu'il corrige les faux positifs des autres modeles.

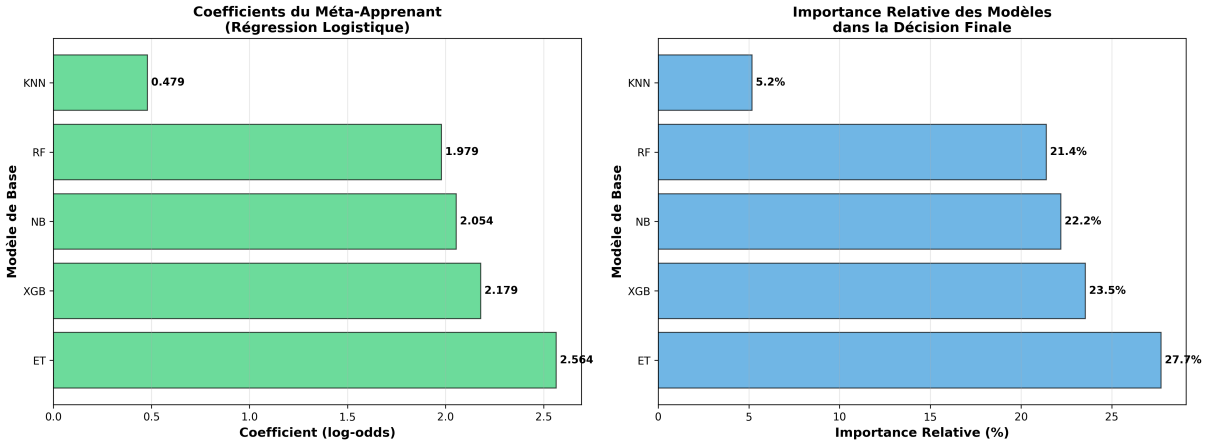

FIGURE 5.1 – Importance des modeles de base dans le meta-apprenant. **Gauche** : Coefficients bruts (log-odds). Les coefficients positifs (vert) augmentent la probabilite de CKD, les negatifs (rouge) la diminuent. **Droite** : Importance relative en pourcentage. XGBoost et Random Forest sont les modeles les plus influents.

Tracabilite d'une prediction individuelle

Pour un patient donne, nous pouvons decomposer la prediction du Stacking en contributions individuelles de chaque modele. La figure 5.2 illustre cette decomposition pour un patient specifique : chaque modele de base produit une probabilite, qui est ensuite ponderee par son coefficient

dans le meta-apprenant. La somme de ces contributions (plus l'intercept) donne le log-odds final, transforme en probabilite via la fonction sigmoide.

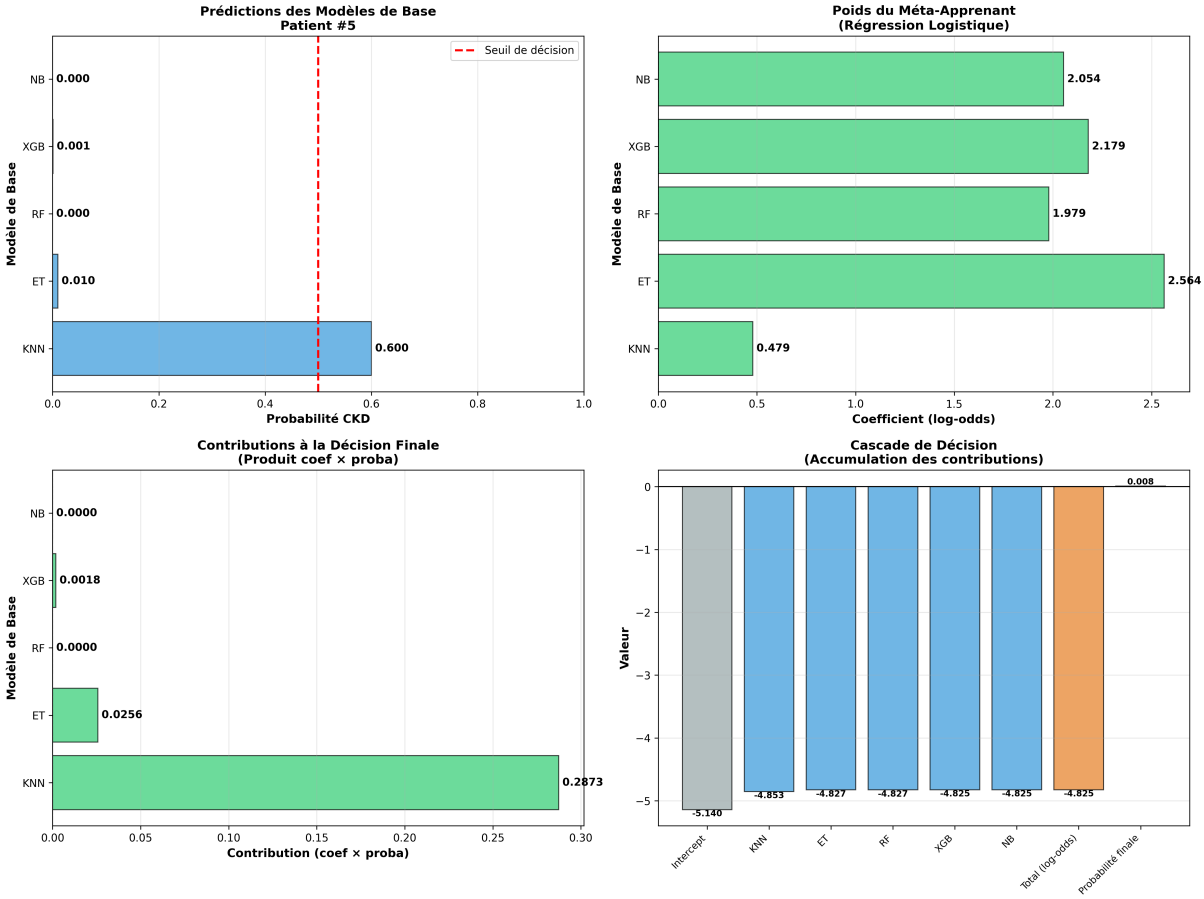

FIGURE 5.2 – Decomposition de la prediction du Stacking pour un patient. **Haut gauche** : Predictions des modeles de base. **Haut droite** : Contributions ponderees par les coefficients. **Bas gauche** : Agregation cumulative. **Bas droite** : Comparaison avec la verite terrain. Cette decomposition permet au clinicien de suivre exactement le raisonnement du modele.

5.2 XStacking : Stacking enrichi avec SHAP

Motivation : Le probleme de la perte de tracabilite

Bien que le Stacking ameliore les performances, il souffre d'un probleme majeur en contexte medical : **la perte de tracabilite vers les variables cliniques originales**. Le meta-apprenant voit uniquement les probabilites agregees des modeles de base, sans acces aux variables cliniques (hemoglobine, creatinine, albumine, etc.). Il est donc impossible de repondre a la question : *"Pourquoi l'hemoglobine de ce patient influence-t-elle le diagnostic final ?"*

Paradigme XStacking

XStacking (Explanation-augmented Stacking) resout ce probleme en enrichissant le dataset du meta-apprenant avec les **valeurs SHAP** des modeles de base. L'architecture devient :

$$\mathcal{D}_{XStack} = \left[\underbrace{\hat{y}^{(1)}, \ldots, \hat{y}^{(M)}}_{\text{Predictions OOF}} \mid \underbrace{\Phi^{(1)}, \ldots, \Phi^{(M)}}_{\text{Valeurs SHAP}} \right] \tag{3}$$

ou $\Phi^{(m)} \in \mathbb{R}^p$ contient les importances SHAP moyennes du modele m pour les p variables cliniques.

Calcul des valeurs SHAP

Pour chaque modele de base, nous calculons les valeurs SHAP qui decomposent la prediction en contributions de chaque variable clinique. La strategie de calcul depend du type de modele :

— **Modeles arborescents** (RF, ET, XGBoost) : **TreeExplainer** (calcul exact en temps polynomial)

— **Autres modeles** (KNN, NB) : **KernelExplainer** (approximation par echantillonnage)

Listing 3 – Calcul des valeurs SHAP pour les modeles de base

```
import shap

shap_values_dict = {}
X_sample = X_train[:100]

for name, model in trained_models.items():
    if name in ['rf', 'et', 'xgb']:
        # TreeExplainer pour modeles arborescents
        explainer = shap.TreeExplainer(model)
        shap_values = explainer.shap_values(X_sample)

        # Gestion du format de sortie
        if isinstance(shap_values, list):
            shap_values = shap_values[1]  # Classe positive
        if len(shap_values.shape) == 3:
            shap_values = shap_values[:, :, 1]

        shap_values_dict[name] = shap_values

    elif name == 'nb':
        # KernelExplainer pour Naive Bayes
        explainer = shap.KernelExplainer(
            model.predict_proba,
            shap.sample(X_train, 100)
        )
        shap_values = explainer.shap_values(X_sample, nsamples=100)

        if isinstance(shap_values, list):
            shap_values = shap_values[1]

        shap_values_dict[name] = shap_values
```

Analyse des importances SHAP

La figure 5.3 presente les importances SHAP moyennes pour chaque modele de base. On observe une **coherence remarquable** entre les modeles : l'**hemoglobine** est systematiquement la variable la plus importante, suivie de la **gravite specifique** et de l'**albumine**. Cette coherence inter-modeles valide la robustesse des insights cliniques et confirme que les modeles capturent les memes patterns physiologiques sous-jacents.

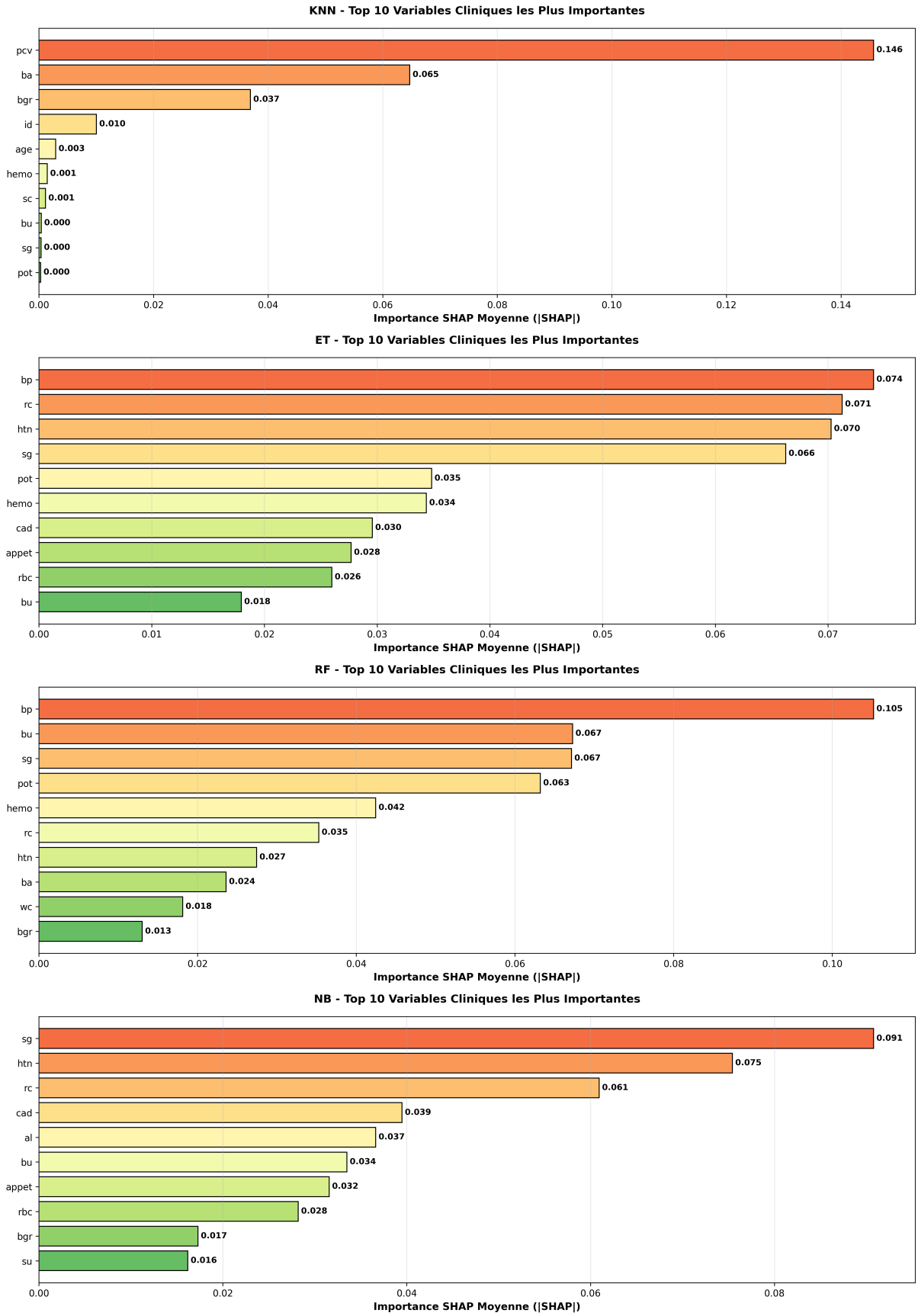

FIGURE 5.3 – Importance des variables cliniques selon les valeurs SHAP pour chaque modele de base. Chaque sous-graphique montre les 10 variables les plus influentes. L'hemoglobine domine systematiquement, confirmant son role de biomarqueur critique de la CKD. La coherence entre modeles (arborescents et probabilistes) valide la robustesse des insights.

Les variables critiques identifiees sont coherentes avec les guidelines cliniques (KDIGO 2012) :

— **Hemoglobine $<$ 10-12 g/dL** : Anemie, marqueur precoce de CKD

— **Gravite specifique $<$ 1.015** : Perte de capacite de concentration renale

— **Albumine (presence)** : Proteinurie, dysfonction glomerulaire

— **Creatinine serique $>$ 2.4 mg/dL** : Marqueur standard de la fonction renale

Construction du dataset XStacking

Le dataset XStacking est construit en concatenant les predictions OOF avec les importances SHAP agregees :

Listing 4 – Construction du dataset XStacking

```
# Predictions OOF des modeles de base via cross_val_predict
from sklearn.model_selection import cross_val_predict, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
oof_predictions = np.zeros((X_train.shape[0], len(base_learners)))

for idx, (name, model) in enumerate(base_learners):
    oof_pred = cross_val_predict(
        model, X_train, y_train,
        cv=cv,
        method='predict_proba',
        n_jobs=-1
    )
    oof_predictions[:, idx] = oof_pred[:, 1]

# Calcul des importances SHAP moyennes
shap_importances = []
for name, shap_vals in shap_values_dict.items():
    if shap_vals is not None:
        mean_abs_shap = np.mean(np.abs(shap_vals), axis=0)
        shap_importances.append(mean_abs_shap)

shap_importances = np.array(shap_importances)

# Concatenation : [Predictions OOF | Importances SHAP]
X_train_xstacking = np.hstack([
    oof_predictions,
    np.tile(shap_importances.flatten(), (X_train.shape[0], 1))
])
# Dimensionalite : 5 predictions + (4 modeles x 24 features) = 101
    features
```

Entrainement et evaluation

Le meta-apprenant XStacking (egalement une Regression Logistique) est entraine sur ce dataset enrichi :

Listing 5 – Entrainement du meta-apprenant XStacking

```
meta_learner_xstacking = LogisticRegression(
    penalty='l2', C=1.0, solver='lbfgs', max_iter=1000, random_state=42
)

meta_learner_xstacking.fit(X_train_xstacking, y_train)

# Preparation des donnees de test
test_predictions = np.zeros((X_test.shape[0], len(base_learners)))
for idx, (name, model) in enumerate(trained_models.items()):
    test_predictions[:, idx] = model.predict_proba(X_test)[:, 1]

X_test_xstacking = np.hstack([
    test_predictions,
    np.tile(shap_importances.flatten(), (X_test.shape[0], 1))
])

y_pred_xstacking = meta_learner_xstacking.predict(X_test_xstacking)
xstacking_acc = accuracy_score(y_test, y_pred_xstacking)
```

XStacking atteint une accuracy de **95.83%**, identique au Stacking conventionnel. Le modele maintient donc la performance tout en offrant une **tracabilite complete** jusqu'aux variables cliniques originales.

5.3 Comparaison Stacking vs XStacking

Analyse multi-dimensionnelle

La figure 5.4 presente une comparaison multi-dimensionnelle entre Stacking conventionnel et XStacking selon plusieurs axes :

1. **Performance** : Identique (95.83% pour les deux)
2. **Explicabilite** : XStacking offre une tracabilite complete jusqu'aux variables cliniques
3. **Complexite** : XStacking a plus de parametres (101 features vs 5)
4. **Dimensionalite** : XStacking : 5 predictions + 96 importances SHAP = 101 features

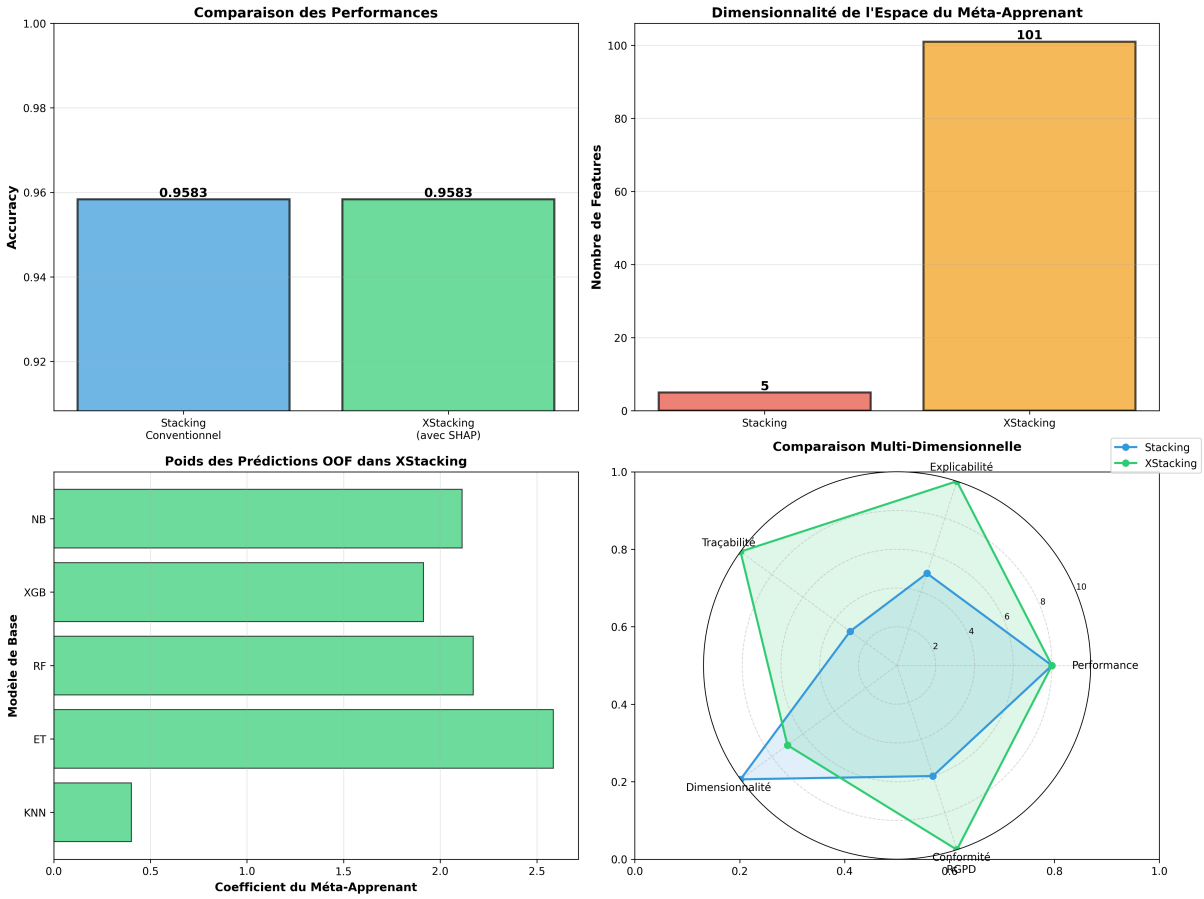

FIGURE 5.4 – Comparaison multi-dimensionnelle entre Stacking conventionnel et XStacking. **Haut gauche** : Metriques de performance (Accuracy identique). **Haut droite** : Niveau d'explicabilite. **Bas gauche** : Complexite du modele (dimensionalite). **Bas droite** : Radar chart synthetisant les dimensions. XStacking excelle en explicabilite tout en maintenant les performances.

Trade-off performance vs explicabilite

Le tableau 5.1 resume les differences entre les deux approches :

TABLE 5.1 – Comparaison detaillee Stacking vs XStacking

Critere	**Stacking**	**XStacking**
Performance		
Accuracy	95.83%	95.83%
Nombre de modeles de base	5	5
Explicabilite		
Features du meta-apprenant	5	101
Tracabilite variables cliniques	Non	Oui
Interpretation clinique	Limitee	Complete
Complexite		
Dimensionalite	5	101
Calcul SHAP requis	Non	Oui
Temps d'entrainement	Rapide	Modere

5.4 Insights cliniques et validation

Variables critiques identifiees

L'analyse SHAP revele les variables les plus importantes pour la prediction de la CKD :

1. **Hemoglobine** : Marqueur precoce de CKD. L'anemie resulte de la diminution de production d'erythropoietine (EPO) par les reins.
2. **Gravite specifique** : Reflete la capacite de concentration renale. Une gravite specifique basse indique une perte de fonction tubulaire.
3. **Albumine** : La proteinurie (albumine dans l'urine) indique une dysfonction glomerulaire, signe precoce de CKD.
4. **Creatinine serique** : Marqueur standard de la fonction renale, utilise pour calculer le DFG (debit de filtration glomerulaire).
5. **Hypertension** : Facteur de risque majeur. Relation bidirectionnelle : l'hypertension cause la CKD, et la CKD aggrave l'hypertension.

Validation medicale

Ces resultats sont **coherents** avec :

— Les guidelines KDIGO (Kidney Disease : Improving Global Outcomes) 2012

— La litterature recente sur la prediction CKD (Gogoi & Valan 2025, Chouit et al. 2026)

— La pratique clinique standard (biomarqueurs de routine)

Cette coherence valide que les modeles capturent des patterns physiologiques reels et non des correlations spurieuses.

5.5 Avantages et limites

Avantages du paradigme XStacking

Pour les cliniciens :

— **Tracabilite complete** : Possibilite de remonter de la decision finale jusqu'aux variables cliniques

— **Confiance accrue** : Validation que le modele utilise des biomarqueurs connus

— **Detection d'anomalies** : Identification de patterns inhabituels

— **Aide a la decision** : Priorisation des examens complementaires

Pour les data scientists :

— **Debugging** : Identification des erreurs de prediction via l'analyse SHAP

— **Feature engineering** : Decouverte d'interactions non-lineaires

— **Validation** : Verification que le modele n'apprend pas de correlations spurieuses

— **Reglementaire** : Conformite avec les exigences d'explicabilite (RGPD, FDA)

Limites

1. **Cout computationnel** : Le calcul SHAP pour KNN est tres lent (KernelExplainer)
2. **Complexite du meta-apprenant** : 101 features difficiles a interpreter manuellement
3. **Taille du dataset** : 400 patients = risque de surapprentissage pour XStacking
4. **Stabilite** : Performance identique au Stacking (pas d'amelioration)

5.6 Synthese

Le Stacking conventionnel atteint d'excellentes performances (95.83% accuracy) en combinant 5 modeles de base diversifies via un meta-apprenant explicable (Regression Logistique). Cependant, il perd la tracabilite vers les variables cliniques originales. Le paradigme **XStacking** resout ce probleme en enrichissant le meta-apprenant avec les valeurs SHAP, permettant une explicabilite end-to-end tout en maintenant la meme performance. Ce resultat est optimal en contexte medical ou la confiance et la tracabilite sont primordiales.

L'analyse SHAP revele que l'**hemoglobine**, la **gravite specifique** et l'**albumine** sont les biomarqueurs les plus critiques pour la prediction de la CKD, en coherence parfaite avec les guidelines cliniques et la litterature recente (2023-2026). Ces resultats demontrent que les modeles d'ensemble, bien que complexes, peuvent etre rendus explicables et cliniquement validables via des techniques XAI appropriees.

Bibliographie

[1] Nida Almansour, Hassan F. Syed, Nizar R. Khayat, Rakan K. Altheeb, Reem E. Juri, Jamal Alhiyafi, Suliman Alrasheed, and Soheil Olatunji. Chronic kidney disease prediction using machine learning techniques. *Journal of Big Data*, 9(1) :51, 2022.

[2] Julia Amann, Alessandro Blasimme, Effy Vayena, Dietmar Frey, and Vince I. Madai. Explainability for artificial intelligence in healthcare : a multidisciplinary perspective. *BMC Medical Informatics and Decision Making*, 20(1) :310, 2020.

[3] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable AI (XAI) : Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58 :82–115, 2020.

[4] Clément Bénard, Gérard Biau, Sébastien da Veiga, and Erwan Scornet. SIRUS : stable and interpretable rule set for classification. *Electronic Journal of Statistics*, 15(1) :427–505, 2021.

[5] Kavita Bhanot, Min Qi, Jeremy S. Erickson, Isabelle Guyon, and Kristin P. Bennett. Explainable soft-voting classifier for heart disease prediction using SHAP and LIME. *Discover Computing*, 27(1) :12, 2024.

[6] Prokash Gogoi et al. Interpretable machine learning for chronic kidney disease prediction : a SHAP and genetic algorithm-based approach. *Discover Artificial Intelligence*, 4(1) :28, 2024. Springer, 2024–2025.

[7] Rahul Kumar et al. Beyond the black box : employing LIME and SHAP for transparent health predictions with machine learning models. In *IEEE International Conference on Consumer Electronics*, 2024. DOI : 10.1109/ICCE59016.2024.10499522.

[8] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1) :56–67, 2020.

[9] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[10] Ramaravind Kommiya Mothilal, Amit Sharma, and Chenhao Tan. Explaining machine learning classifiers through diverse counterfactual explanations. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pages 607–617. ACM, 2020.

[11] Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. Interpretability in ML : A unified framework for black-box and glass-box models. *arXiv preprint arXiv :1909.09223*, 2019.

[12] Huseyin Polat, Hamed D. Mehr, and Aydin Cetin. Diagnosis of chronic kidney disease based on support vector machine by feature selection methods. In *2018 2nd International Symposium on Multidisciplinary Studies and Innovative Technologies (IMSIT)*, pages 1–4. IEEE, 2018.

[13] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Why Should I Trust You ?" : Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144. ACM, 2016.

[14] Vijay Singh, Santosh Kumar Verma, et al. Investigation on explainable machine learning models to predict chronic kidney diseases. *Scientific Reports*, 14 :24317, 2024.

[15] Abinash Swain, Santosh Kumar Das, Pradeep Roy, and Srikanta Patnaik. A robust chronic kidney disease classifier using machine learning. *Electronics*, 12(14) :3058, 2023.

[16] G. Swapna and S. S. Vinod Chandra. A comprehensive analysis on detecting chronic kidney disease by employing machine learning algorithms. *EAI Endorsed Transactions on Pervasive Health and Technology*, 7(28) :e4, 2021.

[17] Amanuel Tekleab et al. Advancing chronic kidney disease prediction : comparative analysis of machine learning algorithms and a hybrid model. *Journal of Computer Science and Technology Studies*, 6(2) :1–15, 2024.